

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

ОТЧЕТ О ВЫПОЛНЕНИИ ПРАКТИЧЕСКОЙ РАБОТЫ №3
по дисциплине «Сети ЭВМ и телекоммуникации компьютерных
сетей»

Вариант 14

Выполнил студент группы
№5130201/30101

Радионова П. В. _____

Проверил преподаватель

Мулюха В. А. _____

«_____» _____ 2026г.

Санкт-Петербург, 2026

1 Постановка задачи

1.1 Цель работы

Разработать и эмулировать распределенную систему, состоящую из сети датчиков-отправителей и процесса-получателя, с использованием контейнеризации Docker и брокера сообщений RabbitMQ.

1.2 Условие задачи

Необходимо эмулировать работу сети датчиков-отправителей и процесса-получателя. Датчики с заданным интервалом времени отправляют сообщения о своем состоянии. При этом необходимо организовать, чтобы получатель имел данные только о текущем (самом актуальном) состоянии отправителей.

1.3 Требования к реализации

- Использовать Docker для контейнеризации всех компонентов системы;
- Применить брокер сообщений RabbitMQ для обмена данными;
- Реализовать механизм хранения только последнего сообщения от каждого датчика.

1.4 Используемые технологии

- **Docker** — платформа для контейнеризации приложений;
- **Docker Compose** — инструмент для определения и запуска многоконтейнерных приложений;
- **RabbitMQ** — брокер сообщений, реализующий протокол AMQP;
- **Python** — язык программирования, используемый для реализации логики датчиков и получателя;
- **Pika** — клиентская библиотека Python для работы с RabbitMQ.

2 Выполнение работы

2.1 Архитектура разрабатываемой системы

Система состоит из пяти контейнеров (таблица 1): одного брокера RabbitMQ, трёх датчиков-отправителей и одного процесса-получателя.

Таблица 1: Компоненты системы

Компонент	Описание
RabbitMQ Broker	Центральный брокер сообщений, обеспечивающий обмен данными между датчиками и получателем
sensor1	Датчик, отправляющий координаты каждые 2 секунды
sensor2	Датчик, отправляющий координаты каждые 3 секунды
sensor3	Датчик, отправляющий координаты каждые 5 секунд
receiver	Процесс-получатель, опрашивающий очереди каждые 5 секунд и выводящий снимок актуального состояния

Каждый датчик имитирует движущийся объект: при каждой отправке его координаты (x, y) случайно смещаются в диапазоне от 0 до 100. Каждому датчику выделена собственная очередь RabbitMQ с именем `last_state_sensorX`.

2.2 Механизм хранения только последнего сообщения

Ключевое требование задания — получатель должен иметь доступ *только к самому актуальному* состоянию каждого датчика. Это реализовано через ограничение размера очереди средствами самого RabbitMQ.

При объявлении очереди задаются два параметра:

- `x-max-length: 1` — очередь хранит не более одного сообщения;
- `x-overflow: drop-head` — при появлении нового сообщения старое (голова очереди) автоматически удаляется.

Таким образом, в каждой очереди физически всегда находится не более одного сообщения — самого последнего, опубликованного датчиком. Предыдущие состояния не накапливаются.

2.3 Режим опроса получателя (pull-модель)

Получатель работает в режиме периодического опроса (*polling*): каждые 5 секунд он последовательно обращается к очереди каждого датчика методом `basic_get` и считывает одно сообщение. После считывания сообщение подтверждается (`ack`) и удаляется из очереди, освобождая место для следующего.

Результат выводится в виде единого снимка состояния (*snapshot*) всех датчиков на текущий момент. Такой подход принципиально отличается от push-модели (`basic_consume`), при которой брокер сам проталкивает каждое входящее сообщение и получатель выводил бы все состояния подряд.

2.4 Структура проекта

```
1 sensor-system/  
2   docker-compose.yaml  
3   sensor/  
4     Dockerfile  
5     requirements.txt  
6     sensor.py  
7   receiver/  
8     Dockerfile  
9     requirements.txt  
10    receiver.py
```

Листинг 1: Структура проекта

2.5 Конфигурация Docker Compose

Файл `docker-compose.yaml` определяет все сервисы системы. Сервисы датчиков и получателя объявлены с условием `depends_on: condition: service_healthy`, что гарантирует их запуск только после успешной проверки готовности RabbitMQ.

```
1 services:  
2   rabbitmq:  
3     image: rabbitmq:3.13-management  
4     container_name: rabbitmq_broker  
5     ports:  
6       - "5672:5672"  
7       - "15672:15672"  
8     environment:  
9       RABBITMQ_DEFAULT_USER: user  
10      RABBITMQ_DEFAULT_PASS: password  
11     healthcheck:  
12       test: ["CMD", "rabbitmq-diagnostics", "ping"]  
13       interval: 5s  
14       timeout: 3s
```

```

15     retries: 5
16
17 sensor1:
18     build: ./sensor
19     container_name: sensor_1
20     environment:
21         SENSOR_ID: "sensor1"
22         RABBITMQ_HOST: rabbitmq
23         RABBITMQ_USER: user
24         RABBITMQ_PASS: password
25         INTERVAL_SEC: 2
26     depends_on:
27         rabbitmq:
28             condition: service_healthy
29
30 sensor2:
31     build: ./sensor
32     container_name: sensor_2
33     environment:
34         SENSOR_ID: "sensor2"
35         RABBITMQ_HOST: rabbitmq
36         RABBITMQ_USER: user
37         RABBITMQ_PASS: password
38         INTERVAL_SEC: 3
39     depends_on:
40         rabbitmq:
41             condition: service_healthy
42
43 sensor3:
44     build: ./sensor
45     container_name: sensor_3
46     environment:
47         SENSOR_ID: "sensor3"
48         RABBITMQ_HOST: rabbitmq
49         RABBITMQ_USER: user
50         RABBITMQ_PASS: password
51         INTERVAL_SEC: 5
52     depends_on:
53         rabbitmq:
54             condition: service_healthy
55
56 receiver:
57     build: ./receiver
58     container_name: receiver
59     environment:
60         RABBITMQ_HOST: rabbitmq
61         RABBITMQ_USER: user
62         RABBITMQ_PASS: password
63     depends_on:
64         rabbitmq:
65             condition: service_healthy

```

2.6 Реализация датчика-отправителя

Каждый датчик представляет собой Python-скрипт, который:

1. Ожидает запуска RabbitMQ (задержка 10 секунд + цикл переподключения);
2. Устанавливает AMQP-соединение с брокером;
3. Объявляет очередь с параметрами `x-max-length=1` и `x-overflow=drop-head`;
4. В бесконечном цикле генерирует случайное смещение координат и публикует JSON-сообщение.

```
1 import os, time, json, random, pika
2
3 SENSOR_ID = os.getenv("SENSOR_ID", "sensor_unknown")
4 RABBITMQ_HOST = os.getenv("RABBITMQ_HOST", "localhost")
5 RABBITMQ_USER = os.getenv("RABBITMQ_USER", "user")
6 RABBITMQ_PASS = os.getenv("RABBITMQ_PASS", "password")
7 INTERVAL_SEC = int(os.getenv("INTERVAL_SEC", "2"))
8 QUEUE_NAME = f"last_state_{SENSOR_ID}"
9
10 x = random.uniform(0.0, 100.0)
11 y = random.uniform(0.0, 100.0)
12
13 def main():
14     global x, y
15     time.sleep(10)
16     credentials = pika.PlainCredentials(RABBITMQ_USER,
17                                         RABBITMQ_PASS)
18     connection = None
19     while connection is None:
20         try:
21             connection = pika.BlockingConnection(
22                 pika.ConnectionParameters(
23                     host=RABBITMQ_HOST, credentials=credentials,
24                     heartbeat=600, blocked_connection_timeout=
25                         300))
26         except pika.exceptions.AMQPConnectionError:
27             time.sleep(5)
28
29     channel = connection.channel()
30     # x-max-length=1 + x-overflow=drop-head:
31     # при поступлении нового сообщения старое вытесняется
32     channel.queue_declare(queue=QUEUE_NAME, durable=True,
```

```

31         arguments={"x-max-length": 1, "x-overflow": "drop-head"})
32
33     while True:
34         x = max(0.0, min(100.0, x + random.uniform(-2.0, 2.0)))
35         y = max(0.0, min(100.0, y + random.uniform(-2.0, 2.0)))
36         state = {"sensor_id": SENSOR_ID, "timestamp": time.time()
37                 ,
38                 "x": round(x, 2), "y": round(y, 2)}
39         channel.basic_publish(
40             exchange="", routing_key=QUEUE_NAME,
41             body=json.dumps(state),
42             properties=pika.BasicProperties(delivery_mode=2))
43         print(f"[{SENSOR_ID}] Sent: x={x:.2f}, y={y:.2f}")
44         time.sleep(INTERVAL_SEC)
45
46 if __name__ == "__main__":
47     main()

```

Листинг 3: sensor.py

2.7 Реализация получателя

Получатель работает в режиме *polling*: каждые 5 секунд он считывает по одному сообщению из каждой очереди (метод `basic_get`) и выводит общий снимок состояния системы.

```

1  import os, json, pika, time
2
3  RABBITMQ_HOST = os.getenv("RABBITMQ_HOST", "localhost")
4  RABBITMQ_USER = os.getenv("RABBITMQ_USER", "user")
5  RABBITMQ_PASS = os.getenv("RABBITMQ_PASS", "password")
6  POLL_INTERVAL = int(os.getenv("POLL_INTERVAL", "5"))
7  SENSOR_IDS = ["sensor1", "sensor2", "sensor3"]
8
9  def get_latest_states(channel):
10     states = {}
11     for sid in SENSOR_IDS:
12         method, props, body = channel.basic_get(
13             queue=f"last_state_{sid}", auto_ack=True)
14         if body:
15             states[sid] = json.loads(body)
16     return states
17
18 def print_snapshot(states):
19     ts = time.strftime("%H:%M:%S")
20     print(f"\n{'='*46}")
21     print(f"    SNAPSHOT at {ts}")
22     print(f"{'='*46}")
23     print(f"    {'Sensor':<12} {'x':>8} {'y':>8} {'Age(s)':>8}")
24     print(f"    {'-'*12} {'-'*8} {'-'*8} {'-'*8}")
25     for sid in SENSOR_IDS:

```

```

26     if sid in states:
27         d = states[sid]
28         age = round(time.time() - d["timestamp"], 1)
29         print(f"    {sid:<12} {d['x']:>8.2f} {d['y']:>8.2f}"
30               f"    {age:>7.1f}s")
31     else:
32         print(f"    {sid:<12} {'--':>8} {'--':>8} {'no data
33               ':>8}")
34     print(f"{'='*46}")
35
36 def main():
37     time.sleep(10)
38     credentials = pika.PlainCredentials(RABBITMQ_USER,
39                                         RABBITMQ_PASS)
40     connection = None
41     while connection is None:
42         try:
43             connection = pika.BlockingConnection(
44                 pika.ConnectionParameters(
45                     host=RABBITMQ_HOST, credentials=credentials,
46                     heartbeat=600, blocked_connection_timeout=
47                         =300))
48         except pika.exceptions.AMQPConnectionError:
49             time.sleep(5)
50
51     channel = connection.channel()
52     for sid in SENSOR_IDS:
53         channel.queue_declare(
54             queue=f"last_state_{sid}", durable=True,
55             arguments={"x-max-length": 1, "x-overflow": "drop-
56                       head"})
57
58     while True:
59         print_snapshot(get_latest_states(channel))
60         time.sleep(POLL_INTERVAL)
61
62 if __name__ == "__main__":
63     main()

```

Листинг 4: receiver.py

2.8 Dockerfile для датчика и получателя

Оба компонента используют одинаковый минимальный образ Python:

```

1 FROM python:3.11-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY sensor.py .
6 CMD ["python", "sensor.py"]

```

Листинг 5: Dockerfile датчика

```
1 FROM python:3.11-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY receiver.py .
6 CMD ["python", "receiver.py"]
```

Листинг 6: Dockerfile получателя

Файл `requirements.txt` для обоих компонентов содержит единственную зависимость:

```
1 pika
```

2.9 Последовательность запуска

2.9.1 Шаг 1: Подготовка окружения

1. Установить Docker Desktop для Windows;
2. Убедиться в наличии WSL 2 и включённой аппаратной виртуализации;
3. Разместить файлы проекта в соответствии со структурой, описанной выше.

2.9.2 Шаг 2: Сборка и запуск

Выполнить в корне проекта:

```
1 docker compose up --build
```

2.9.3 Шаг 3: Наблюдение за работой

В терминале отображаются логи всех контейнеров. Датчики публикуют координаты с заданными интервалами. Каждые 5 секунд получатель выводит единый снимок актуального состояния всех датчиков.

3 Результаты работы

3.1 Вывод датчиков

Каждый датчик при запуске выводит в лог подтверждение отправки:

```
1 sensor_1 | [sensor1] Waiting for RabbitMQ to start...
2 sensor_2 | [sensor2] Waiting for RabbitMQ to start...
3 sensor_3 | [sensor3] Waiting for RabbitMQ to start...
4 sensor_1 | [sensor1] Started, sending every 2s to queue '
    last_state_sensor1'
5 sensor_1 | [sensor1] Sent: x=47.31, y=62.18
6 sensor_2 | [sensor2] Started, sending every 3s to queue '
    last_state_sensor2'
7 sensor_2 | [sensor2] Sent: x=83.05, y=21.44
8 sensor_3 | [sensor3] Started, sending every 5s to queue '
    last_state_sensor3'
9 sensor_3 | [sensor3] Sent: x=15.60, y=77.93
10 sensor_1 | [sensor1] Sent: x=49.02, y=60.55
```

Листинг 7: Пример вывода датчиков

3.2 Вывод получателя

Получатель каждые 5 секунд выводит единый снимок — только *текущее* состояние каждого датчика, без истории:

```
1 receiver | [RECEIVER] Polling latest state every 5s. Press CTRL+
    C to stop.
2
3 receiver | =====
4 receiver |     SNAPSHOT at 14:32:07
5 receiver | =====
6 receiver |     Sensor           x           y       Age(s)
7 receiver |     -----
8 receiver |     sensor1         49.02       60.55       1.3s
9 receiver |     sensor2         83.05       21.44       0.7s
10 receiver |     sensor3         15.60       77.93       3.1s
11 receiver |     =====
12
13 receiver | =====
14 receiver |     SNAPSHOT at 14:32:12
15 receiver | =====
16 receiver |     Sensor           x           y       Age(s)
17 receiver |     -----
18 receiver |     sensor1         50.88       58.71       0.4s
19 receiver |     sensor2         81.33       23.10       2.1s
20 receiver |     sensor3         15.60       77.93       8.1s
21 receiver |     =====
```

Листинг 8: Пример вывода получателя

Видно, что координаты датчиков изменяются от снимка к снимку — каждый раз отображается только самое актуальное положение. Поле `Age(s)` показывает, сколько секунд прошло с момента публикации последнего сообщения датчиком, что позволяет оценить актуальность данных.

4 Заключение

В ходе выполнения лабораторной работы была разработана и развёрнута распределённая система, эмулирующая работу сети датчиков-отправителей и процесса-получателя.

Для обеспечения требования о доступности только актуального состояния использован механизм ограничения длины очереди RabbitMQ: параметры `x-max-length=1` и `x-overflow=drop-head` гарантируют, что в каждой очереди физически хранится не более одного сообщения. Получатель реализован в режиме периодического опроса (`basic_get`), что позволяет получать именно снимок текущего состояния, а не непрерывный поток всех поступивших сообщений.

Все компоненты системы упакованы в Docker-контейнеры и описаны в едином файле `docker-compose.yaml`, что обеспечивает воспроизводимость развёртывания. Зависимости между сервисами управляются через механизм `healthcheck`, исключая запуск датчиков до готовности брокера.